AN2DL - First Challenge Report
Backpropagandisti

Giovanni Passuello, Gabriele Proverbio, Piero Burigana, Matteo Trombetta
giovannipassuello, gabrieleproverbio, pieroburigana, matteotrombetta
278147, 273983, 260131, 279729

Abstract

This report describes the `GRU_MONO` notebook (see Section 6) developed for the *Pirates Pain* challenge of the "Artificial Neural Networks and Deep Learning" course. The model combines recurrent neural networks with learned embeddings for categorical features and a temporal attention mechanism. A pirate-based 5-fold cross-validation is used to ensure user-level separation. Although this notebook is not the one that achieved the top Kaggle score (0.9554), it proved to be the most stable and reproducible solution, with user-level F1 scores consistently above 0.95. These findings highlight the robustness of embedding-based feature modelling and attention-driven temporal processing.

1 Task and Data Representation

The task is unchanged: predict the overall pain label of a pirate (`no_pain`, `low_pain`, `high_pain`) from a multivariate time series of length T plus static information.

Each pirate is represented by:

- dynamic features recorded over time: joint-related variables `joint_k` and survey answers `pain_survey_1-4`;
- static attributes: `n_legs`, `n_hands`, `n_eyes`.

Labels are provided in a separate file and mapped to integers:

$$\texttt{no_pain} \to 0, \quad \texttt{low_pain} \to 1, \quad \texttt{high_pain} \to 2.$$

As before, the evaluation is at *pirate level*, not at timestep: all windows belonging to a pirate share the same target.

2 Preprocessing and Feature Engineering

2.1 Cleaning and encoding of categorical features

The notebook first standardises the static "limb" information, mapping the string values to integer counts:

$$\texttt{two} \mapsto 2, \qquad \texttt{one+*} \mapsto 1,$$

After replacement, these columns are cast to integer type in both `X_train` and `X_test` to be compatible with `nn.Embedding`.

Similarly, the pain survey features `pain_survey_1--4` are explicitly cast to integer and treated as categorical indices for embedding layers.

2.2 Merging labels and basic checks

The training features and labels are merged via `sample_index`:

$$X_\text{train_labeled} = \texttt{merge}(X_\text{train}, y_\text{train}, \texttt{on='sample_index'})$$

Exploratory analysis includes:

- distributions of `n_legs`, `n_hands`, `n_eyes` across pain categories;
- distributions of the survey answers `pain_survey_k`;
- average joint trajectories over time for each pain label.

This EDA justifies treating limbs and survey answers as categorical, and joints as continuous signals.

2.3 Scaling of numerical features

Joint features form the numerical part of the input; limb and survey columns are *not* scaled because

they feed embedding layers.

For the recurrent part, the notebook uses a robust-style scaling based on median and interquartile range (IQR), computed on the training data. The idea is:

$$x' = \frac{x - \text{median}}{\text{IQR}},$$

with a stability check to deal with constant features. This transformation is applied fold-by-fold during cross-validation and later on the test set.

2.4 Windowing and sequence construction

To feed the recurrent model, the full time series of each pirate is split into overlapping windows. The final configuration in this notebook uses:

$$\text{window size } W = 32, \qquad \text{stride } S = 16,$$

values selected based on previous experimentation (they offer a good compromise between local context and number of training examples).

A function `build_sequences_with_embeddings` loops over each `sample_index`, sorts its rows by `time` and extract three aligned sequences:

1. pain survey sequence: $\mathbf{x}^{(ps)} \in \mathbb{Z}^{T\times 4}$,
2. limb sequence: $\mathbf{x}^{(limb)} \in \mathbb{Z}^{T\times 3}$,
3. numerical joint sequence: $\mathbf{x}^{(num)} \in \mathbb{R}^{T\times F}$,

then slides a window of length W with stride S, producing many triplets $(\mathbf{x}_j^{(ps)}, \mathbf{x}_j^{(limb)}, \mathbf{x}_j^{(num)})$ per pirate; In the end, assigns the pirate label to each window and keeps track of the corresponding `sample_index`.

This yields three NumPy arrays (pain survey, limbs, numerical features) plus two vectors (labels and sample indices), later wrapped into `TensorDataset` and `DataLoader` objects.

3 Model: GRU with Embeddings and Attention

3.1 Overall structure

The model is a recurrent classifier implemented in PyTorch, combining:

- joint features (numeric, normalised);
- categorical inputs, embedded separately;
- a GRU encoder over windows of W=32 timesteps;
- a temporal attention layer to aggregate hidden states.

Key hyperparameters:

- GRU (2 layers, hidden size 128, unidirectional)
- Embeddings: 8-dim per categorical feature
- Dropout: 0.4
- Classes: 3 (softmax)

The attention mechanism learns a weighted sum over hidden states and allows the model to focus on the most informative moments in each window.

3.2 Embedding blocks

Two groups of embedding layers are defined:

1. **Pain survey embeddings:** a `ModuleList` of 4 separate `nn.Embedding` layers, one per `pain_survey_k`
2. **Limb embeddings:** a second `ModuleList` of 3 `nn.Embedding` layers, one for each of `n_legs`, `n_hands`, `n_eyes`. The dictionary size is set to 3 (indices 0, 1, 2), even if only 1 and 2 are used.

At each timestep, the embeddings for all pain survey items are concatenated, producing a dense vector summarising the "subjective" pain component. Similarly, the three limb embeddings are concatenated, capturing the static physical condition of the pirate in a dense, learnable way.

These vectors are then concatenated with the numerical joint features for that timestep:

$$\mathbf{z}_t = [\,\text{embed_pain}(t) \parallel \text{embed_limbs}(t) \parallel \mathbf{x}_t^{(num)}\,],$$

yielding the input for the GRU at time t.

3.3 Temporal attention

Instead of using only the last hidden state of the GRU, the notebook defines an `Attention` module. Given the sequence of hidden states $\mathbf{H} \in \mathbb{R}^{B\times W\times H}$ (batch, window length, hidden size), attention works as:

1. apply a learned linear transformation

$$\mathbf{U} = \tanh(\mathbf{H}W),$$

2. project $\mathbf{U}$ on a learned context vector $\mathbf{v}$ to obtain scores over timesteps;
3. apply a softmax over time to get attention weights α_t;

4. compute a context vector as weighted sum:

$$\mathbf{c} = \sum_t \alpha_t \mathbf{h}_t.$$

The context vector $\mathbf{c}$ summarises the whole window, emphatising timesteps that are more informative for the classification. Finally, a linear classifier maps $\mathbf{c}$ to logits for the three pain classes.

4 Training Procedure and 5-fold Cross-Validation

4.1 Loss, regularisation and optimisation

The loss function is a `CrossEntropyLoss` with:

- **class weights** computed via compute_class_weight on the label distribution, then smoothed with a factor $a = 0.7$ to avoid aggressive reweighting;
- **label smoothing** with parameter 0.02, regularising over-confident predictions.

Training uses:

- optimiser: AdamW with learning rate 10^{-4} and L2 penalty $\lambda_2 = 10^{-3}$;
- L1 penalty $\lambda_1 = 0$
- gradient clipping with max norm 1.0;
- mixed precision (`torch.amp.GradScaler`) when running on GPU.

4.2 Pirate-based 5-fold validation

The function k_shuffle_split_cross_validation_round_rnn performs 5-fold cross-validation, shuffling the unique `sample_index` values and selecting, for each split, a subset of 66 of them to be part of the validation set. Then, for both train and validation folds, it builds windows, using `build_sequences_with_embeddings`, and tensors. In the end, it builds the needed `DataLoader`s and trains the model up to 500 epochs, with an early stopping mechanism on validation F1Score to limit overfitting.

During training, `train_one_epoch_emb` computes F1 at window level for tracking;`validate_one_epoch_emb` aggregates probabilities per pirate (averaging the softmax outputs across its windows) and then computes F1 at user level. The F1 used internally is a weighted F1, which correlates well with macro F1 on this dataset.

For each split, the best validation F1 over epochs is stored. At the end, the function reports the mean and standard deviation of the best scores across the 5 splits.

5 Results and Discussion

The main outcome of this notebook is that the GRU with embeddings and attention achieves **very stable performance across the 5 splits**, with user-level F1 consistently high: **validation score: 0.9459±0.0132**

In other words, the model is not only accurate but also reliable across different subsets of pirates, which is crucial for a replicable model.

The key factors that likely contribute to this behaviour are the explicit modelling of categorical information through embeddings, that allows the model to learn flexible representations of these variables, the use of temporal attention, that lets the model focus more on best informative models, the splitting strategy, that avoids data leakage through different folds, and the use of class weights and label smoothing, to improve performance on minority classes and gain stability:

6 F1-Score Comparison

Model	F1-Score
HIGHEST_SCORE*	(Kaggle) 0.9554
GRU_BI	(Kaggle) 0.931
GRU_MONO	**0.9463 ± 0.0104**

Note: `GRU_MONO` was selected as the final model due to its superior generalization and overall performance.

7 Conclusion

This work presented a self-contained modelling pipeline based on embeddings, attention, and user-level cross-validation, yielding stable and strong performance on the *Pirates Pain* challenge. In addition to the notebook described here, several alternative approaches were explored during development (including focal loss, feature augmentation, oversampling strategies and convolutional layers). Due to space constraints and different methodological focus, they are not included in this report. However, these additional experiments are fully documented in the supplementary notebooks included with this submission.

References

[1] scikit-learn Developers, *sklearn.utils.class_weight.compute_class_weight — scikit-learn Documentation*, Available at: https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html (Accessed: 2025).

[2] scikit-learn Developers, *sklearn.preprocessing.RobustScaler — scikit-learn Documentation*, Available at: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html (Accessed: 2025).

[3] scikit-learn Developers, *scikit-learn Documentation*, Available at: https://scikit-learn.org/stable/documentation.html (Accessed: 2025).

[4] NumPy Developers, *NumPy Documentation*, Available at: https://numpy.org/doc/ (Accessed: 2025).

[5] Pandas Development Team, *Pandas Documentation*, Available at: https://pandas.pydata.org/docs/ (Accessed: 2025).

[6] Matplotlib Developers, *Matplotlib Documentation*, Available at: https://matplotlib.org/stable/contents.html (Accessed: 2025).

[7] PyTorch Team, *Automatic Mixed Precision (AMP) — PyTorch Documentation*, Available at: https://docs.pytorch.org/stable/amp.html (Accessed: 2025).